UNIVERSIDAD DON BOSCO

DESARROLLO DE APLICACIONES CON WEB FRAMEWORKS

DWF404 G01T

Actividad #3
Investigación Aplicada

Alumno:

- Cesar Alejandro Lara Franco_____LL202677
- Edwin Daniel Leiva Barrera_____LB250368
- Kevin Jafet Aguilar Franco_____AF250113
- Cesar Armando Melendez Arias_____MA250099

Docente:

Ricardo Antonio Gómez Serrano

Fecha de entrega:
17/04/2026

Índice

Prólogo introductorio de la investigación 3
Definición del problema a investigar 4
¿Qué son los ADF (Application Development Framework)? 5
¿Para qué se utilizan y en qué áreas se aplican los ADF? 6
Cómo se configura un proyecto Spring Boot para utilizar ADF (Dependencias) 7
- **Arquitectura típica de integración Spring Boot – ADF** 7
- **Formas de integración soportadas** 8
- **Configuración del proyecto Spring Boot (REST con ADF)** 8
- **Dependencias principales (Maven)** 9
- **¿Por qué estas dependencias?** 10
- **Configuración de acceso al servicio ADF** 10
- **Configuración del cliente REST (WebClient)** 10
- **Consumo de un servicio REST desde ADF** 11
- **Clase DTO en Spring Boot** 11
- **Servicio consumidor en Spring Boot** 11
- **Controlador expuesto por Spring Boot** 12

Código Ejemplo 13
Interpretación o conclusión sobre ADF 19
Bibliografía 20

Prólogo introductorio de la investigación

En la actualidad, el desarrollo de aplicaciones empresariales se enfrenta a una creciente necesidad de flexibilidad, escalabilidad y rápida integración con sistemas existentes. Las organizaciones que cuentan con plataformas consolidadas como Oracle ADF (Application Development Framework) buscan modernizar sus aplicaciones sin descartar por completo sus inversiones tecnológicas previas.

Por otro lado, Spring Boot se ha posicionado como uno de los frameworks más utilizados en el desarrollo de aplicaciones Java modernas debido a su simplicidad, orientación a microservicios y fuerte adopción en arquitecturas basadas en la nube.

Esta investigación tiene como objetivo analizar la integración entre Spring Boot y Oracle ADF, explorando cómo ambos enfoques pueden coexistir y complementarse dentro de entornos empresariales actuales. Se busca comprender el impacto de ADF hoy en día, su vigencia y las posibilidades técnicas de integración con aplicaciones modernas desarrolladas en Spring.

Definición del problema a investigar

Muchas organizaciones poseen aplicaciones críticas desarrolladas con Oracle ADF, las cuales ofrecen robustez, integración con bases de datos Oracle y un fuerte soporte empresarial. Sin embargo, estas aplicaciones suelen presentar limitaciones en agilidad, escalabilidad y adopción de arquitecturas modernas, como los microservicios.

El problema central de esta investigación es determinar:

¿Cómo puede integrarse Spring Boot con Oracle ADF para aprovechar las ventajas de ambos frameworks sin comprometer la estabilidad ni la evolución tecnológica de las aplicaciones empresariales?

Asimismo, se analiza si ADF continúa siendo una tecnología relevante y cómo puede adaptarse a los nuevos paradigmas de desarrollo.

¿Qué son los ADF (Application Development Framework)?

Oracle Application Development Framework (ADF) es un framework de desarrollo empresarial basado en Java, creado por Oracle, que permite construir aplicaciones web altamente escalables y seguras. Está diseñado siguiendo el patrón MVC (Modelo–Vista–Controlador) y proporciona un conjunto completo de herramientas para el desarrollo rápido de aplicaciones empresariales.

ADF está compuesto por varias capas:

- ADF Business Components (ADF BC):
  Gestiona la lógica de negocio, entidades, vistas y acceso a datos.
- ADF Model:
  Actúa como intermediario entre los componentes del negocio y la interfaz gráfica.
- ADF Controller:
  Administra la navegación y el flujo de la aplicación.
- ADF Faces:
  Basado en JavaServer Faces (JSF), permite crear interfaces web ricas.

Características principales:

- Integración nativa con bases de datos Oracle
- Desarrollo visual mediante Oracle JDeveloper
- Seguridad integrada
- Arquitectura orientada a componentes

¿Para qué se utilizan y en qué áreas se aplican los ADF?

Oracle ADF se utiliza principalmente en entornos empresariales de gran escala, donde se requieren aplicaciones robustas, seguras y bien estructuradas.

Áreas de aplicación más comunes:

- Sistemas ERP y CRM
- Aplicaciones financieras
- Sistemas gubernamentales
- Aplicaciones administrativas internas
- Plataformas corporativas integradas con Oracle SOA

Ventajas:

- Alto nivel de estabilidad
- Integración completa con el ecosistema Oracle
- Ideal para arquitecturas monolíticas empresariales

Desventajas actuales:

- Curva de aprendizaje elevada
- Dependencia fuerte de herramientas propietarias
- Menor flexibilidad frente a microservicios

Cómo se configura un proyecto Spring Boot para utilizar ADF (Dependencias)

Antes de entrar en la configuración técnica, es indispensable aclarar un punto clave desde el punto de vista arquitectónico:

Spring Boot y Oracle ADF no se integran como librerías internas dentro de un mismo proyecto Java, debido a que:

- ADF es un framework monolítico, orientado a aplicaciones empresariales completas
- Spring Boot es un framework ligero, orientado a microservicios y APIs
- ADF depende fuertemente de Oracle JDeveloper, ADF Runtime y WebLogic
- Spring Boot utiliza contenedores embebidos (Tomcat, Jetty, Netty)

Por lo tanto, la integración correcta se realiza a nivel de servicios, no a nivel de código compartido.

Arquitectura típica de integración Spring Boot – ADF

La forma más común y recomendada de integración es la siguiente:

- Oracle ADF
    - Mantiene la lógica de negocio
    - Accede a la base de datos
    - Expone servicios SOAP o REST
- Spring Boot
    - Consume servicios expuestos por ADF
    - Actúa como capa moderna de integración
    - Implementa APIs REST, microservicios o gateways
    - Sirve frontends modernos o aplicaciones móviles

En este escenario:

- ADF = Backend empresarial
- Spring Boot = Capa moderna de servicios

Formas de integración soportadas

Spring Boot puede integrarse con ADF principalmente de tres formas:

1. Consumo de servicios REST expuestos por ADF
2. Consumo de servicios SOAP expuestos por ADF
3. Integración indirecta vía base de datos (menos recomendada)

La mejor práctica actual es la opción REST, por desacoplamiento y escalabilidad.

Configuración del proyecto Spring Boot (REST con ADF)

1. Creación del proyecto Spring Boot

El proyecto se crea como cualquier aplicación Spring Boot estándar, utilizando:

- Spring Initializr
- Maven
- Java 11 o superior

No se requieren librerías de Oracle ADF dentro de Spring Boot.

Dependencias principales (Maven)

Estas dependencias permiten a Spring Boot comunicarse con servicios ADF vía HTTP/REST.

```
<dependencies>

    <!-- Spring Boot Web (REST MVC) -->
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-web</artifactId>
    </dependency>

    <!-- WebClient (REST reactivo, recomendado) -->
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-webflux</artifactId>
    </dependency>

    <!-- Manejo de JSON -->
    <dependency>
        <groupId>com.fasterxml.jackson.core</groupId>
        <artifactId>jackson-databind</artifactId>
    </dependency>

    <!-- Validaciones -->
    <dependency>
        <groupId>jakarta.validation</groupId>
        <artifactId>jakarta.validation-api</artifactId>
    </dependency>

    <!-- Seguridad (opcional, si ADF usa auth) -->
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-security</artifactId>
    </dependency>

</dependencies>
```

¿Por qué estas dependencias?

Dependencia	Función
spring-boot-starter-web	Crear controladores REST
webflux	Consumir servicios REST (ADF)
jackson	Mapear JSON ↔ objetos Java
validation	Validar datos recibidos
security	Manejo de autenticación (si ADF lo requiere)

Configuración de acceso al servicio ADF

Spring Boot necesita conocer la URL base del servicio ADF.

application.yml

```yaml
adf:
  base-url: http://servidor-adf:7001/aplicacion/rest
```

Configuración del cliente REST (WebClient)

```java
@Configuration
public class AdfClientConfig {

    @Value("${adf.base-url}")
    private String adfBaseUrl;

    @Bean
    public WebClient adfWebClient() {
        return WebClient.builder()
                .baseUrl(adfBaseUrl)
                .defaultHeader(HttpHeaders.CONTENT_TYPE, MediaType.APPLICATION_JSON_VALUE)
                .build();
    }
}
``
```

Consumo de un servicio REST desde ADF

```
GET /rest/clientes
```

Clase DTO en Spring Boot

```
1  public class ClienteDTO {
2
3      private Long id;
4      private String nombre;
5      private String email;
6
7      // getters y setters
8  }
9  ``
```

Servicio consumidor en Spring Boot

```
1  @Service
2  public class ClienteService {
3
4      private final WebClient adfWebClient;
5
6      public ClienteService(WebClient adfWebClient) {
7          this.adfWebClient = adfWebClient;
8      }
9
10     public List<ClienteDTO> obtenerClientesDesdeAdf() {
11         return adfWebClient
12                 .get()
13                 .uri("/clientes")
14                 .retrieve()
15                 .bodyToFlux(ClienteDTO.class)
16                 .collectList()
17                 .block();
18     }
19 }
```

Controlador expuesto por Spring Boot

```
1  @RestController
2  @RequestMapping("/api/clientes")
3  public class ClienteController {
4
5      private final ClienteService clienteService;
6
7      public ClienteController(ClienteService clienteService) {
8          this.clienteService = clienteService;
9      }
10
11     @GetMapping
12     public List<ClienteDTO> listarClientes() {
13         return clienteService.obtenerClientesDesdeAdf();
14     }
15 }
```

En este punto:

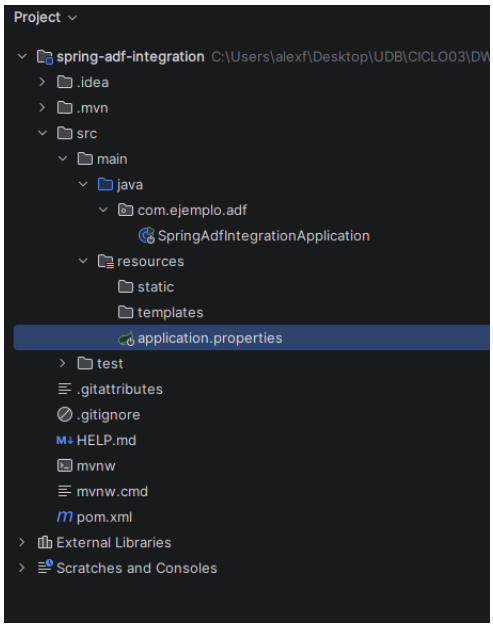
- Spring Boot actúa como intermediario
- ADF sigue controlando la lógica de negocio
- Se evita duplicación de reglas de negocio

Configurar un proyecto Spring Boot para trabajar con ADF no consiste en importar ADF como dependencia, sino en diseñar una arquitectura de integración orientada a servicios. Spring Boot actúa como una capa moderna, flexible y escalable que complementa la robustez de Oracle ADF sin reemplazarlo por completo.

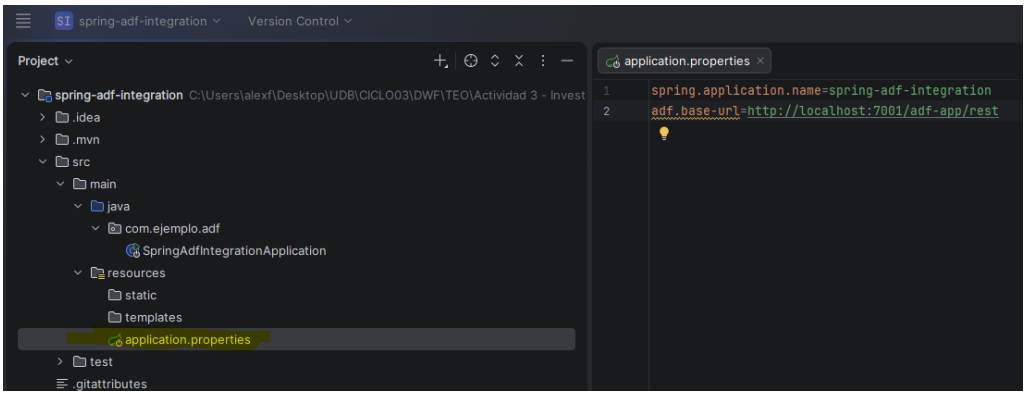
Código Ejemplo

Spring Web
Spring Reactive Web
Validation

Estructura

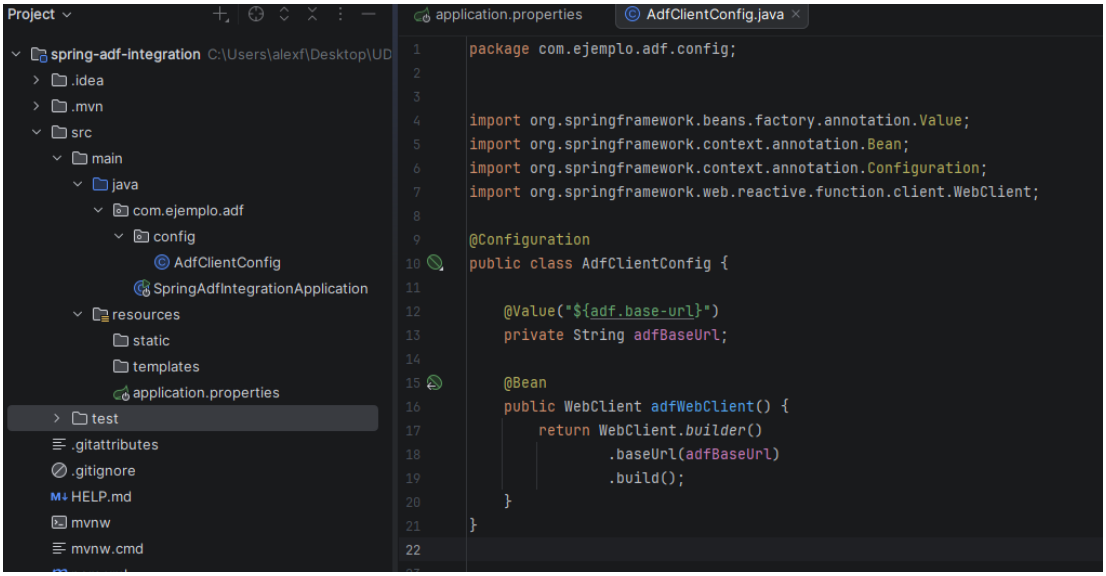


Le dejamos saber a Spring Boot dónde esta ADF



Aquí estamos guardando la dirección del sistema ADF

Creamos Paquete config/Clase AdfClientConfig.java



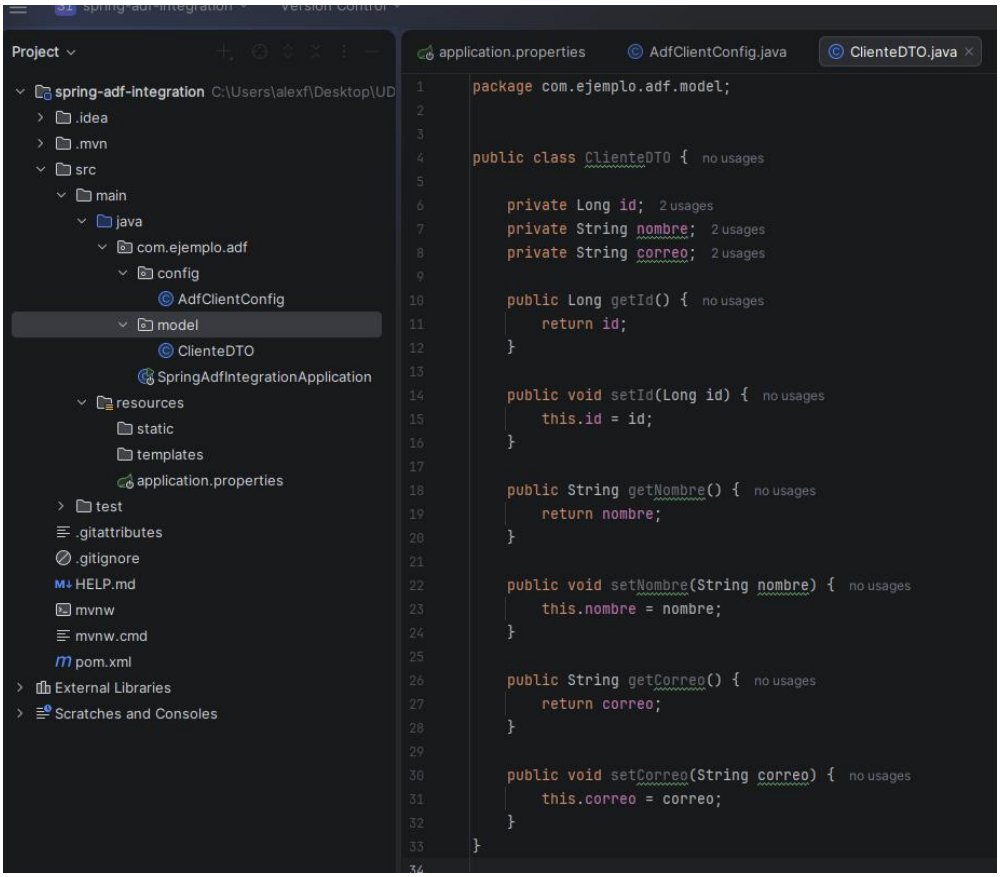
¿Qué hace esta clase?

- Lee la URL del ADF
- Crea un cliente HTTP
- Permite consumir servicios REST

Creamos el modelo de datos (DTO)

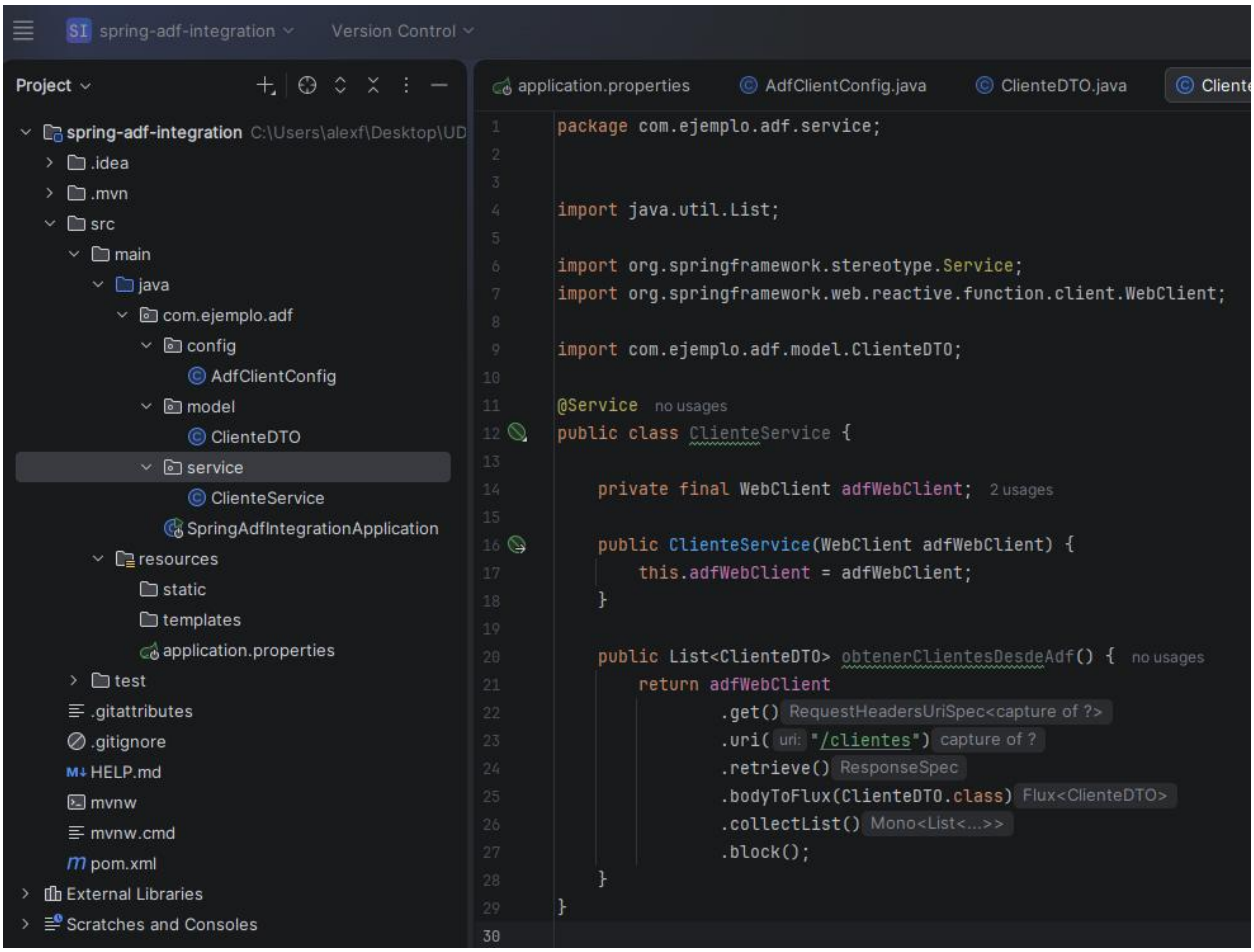
ADF nos devuelve datos JSON.
Necesitamos una clase para representarlos.

Creamos paquete model



Esto representa lo que ADF nos envía.

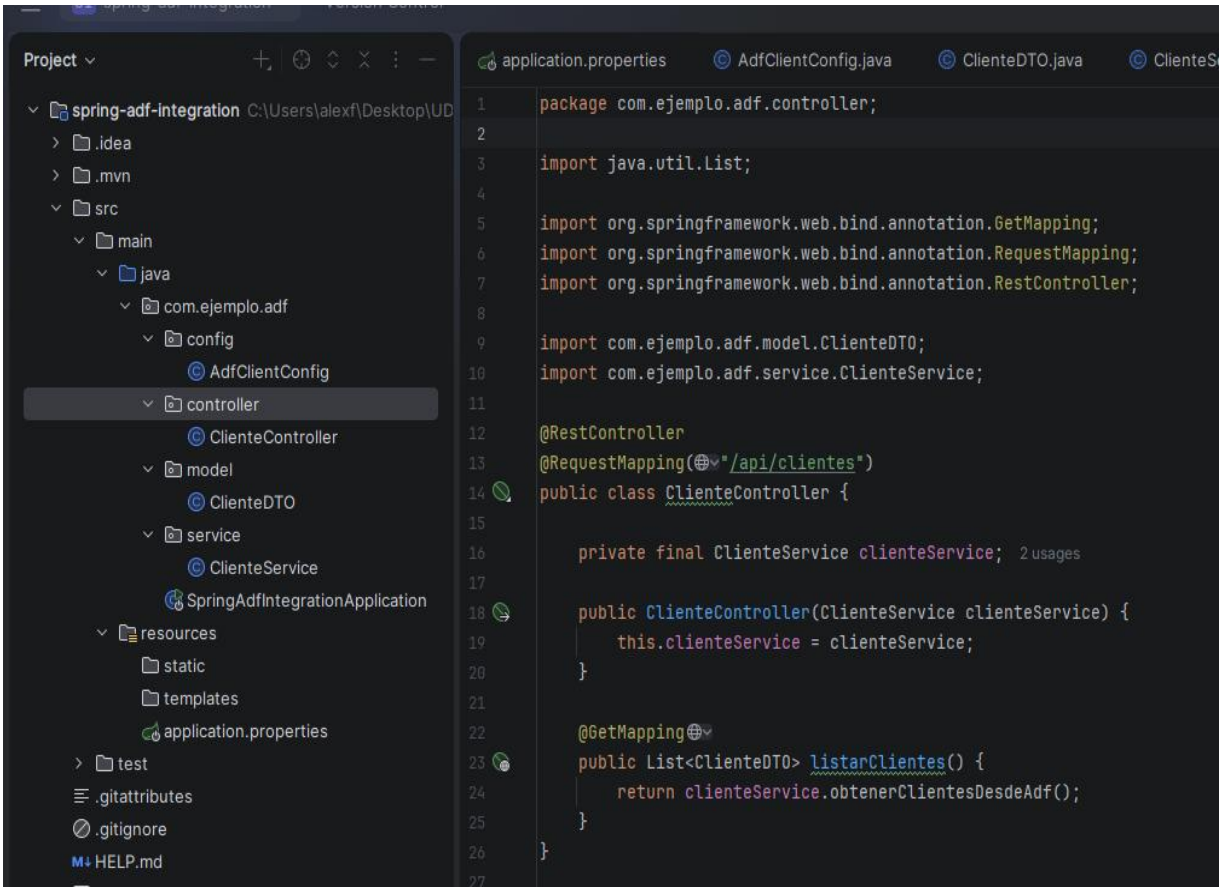
Creamos el servicio que consume ADF



¿Qué hace?

- Llama al ADF
- Lee la lista de clientes
- Devuelve los datos a Spring Boot

Creamos el controlador REST de Spring Boot



Esto crea

GET http://localhost:8080/api/clientes

Durante la ejecución del proyecto Spring Boot se presenta un error interno al intentar consumir el servicio ADF, debido a que el sistema ADF no se encuentra desplegado en el entorno local. Este comportamiento es esperado, ya que la aplicación Spring Boot está configurada para integrarse con un ADF previamente existente, el cual se asume operativo en un entorno real.

```
Run   spring-adf-integration [package]
spring-adf-integration [package]: At 4/16/2026 3:09 PM        6 sec, 425 ms
[INFO]
[INFO] --- spring-boot:3.5.13:repackage (repackage) @ spring-adf-integration ---
[INFO] Replacing main artifact C:\Users\alexf\Desktop\UDB\CICLO03\DWF\TEO\Actividad 3 - Investigacion
[INFO] The original artifact has been renamed to C:\Users\alexf\Desktop\UDB\CICLO03\DWF\TEO\Actividad
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
[INFO] Total time:  5.168 s
[INFO] Finished at: 2026-04-16T15:09:57-06:00
[INFO] ------------------------------------------------------------------------
```

```
.gitattributes
.gitignore
HELP.md
        return clienteService.obtenerClientesDesdeAdf();
    }
}
Run   spring-adf-integration [clean]
spring-adf-integration [clean]: At 4/16/2026 3:09 PM        1 sec, 765 ms
[INFO]   from pom.xml
[INFO] --------------------------------[ jar ]---------------------------------
[INFO]
[INFO] --- clean:3.4.1:clean (default-clean) @ spring-adf-integration ---
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
[INFO] Total time:  0.312 s
[INFO] Finished at: 2026-04-16T15:09:29-06:00
[INFO] ------------------------------------------------------------------------
```

```
ebServer  : Tomcat started on port 8080 (http) with context path '/'
lication  : Started SpringAdfIntegrationApplication in 1.164 seconds (process running for 1.332)
[/]       : Initializing Spring DispatcherServlet 'dispatcherServlet'
et        : Initializing Servlet 'dispatcherServlet'
et        : Completed initialization in 0 ms
rvlet]    : Servlet.service() for servlet [dispatcherServlet] in context with path [] threw exception [Reque
tsockopt: localhost/127.0.0.1:7001] with root cause
```

Interpretación o conclusión sobre ADF

Desde nuestro punto de vista, tanto personal como técnico, se puede decir que Oracle ADF sigue siendo una tecnología fuerte y confiable, especialmente en empresas grandes que llevan muchos años utilizando herramientas de Oracle. Estas empresas han invertido mucho tiempo, dinero y esfuerzo en construir sus sistemas con ADF, por lo que sigue siendo una opción válida para mantener aplicaciones estables y seguras. ADF ofrece una estructura bien definida, buena integración con bases de datos Oracle y herramientas que facilitan el desarrollo de aplicaciones empresariales tradicionales.

Sin embargo, también es importante reconocer que ADF no se adapta tan fácilmente a las nuevas formas de desarrollo, como los microservicios, la nube o las aplicaciones más ligeras. Hoy en día, muchas empresas buscan sistemas más flexibles, rápidos de modificar y fáciles de escalar, y en ese aspecto ADF puede resultar un poco rígido y complejo. Esto ha provocado que su uso en proyectos nuevos sea menor en comparación con otros frameworks más modernos.

Por esta razón, la integración con Spring Boot se presenta como una excelente solución. Spring Boot permite crear aplicaciones modernas, simples y bien organizadas, que pueden comunicarse con sistemas ADF mediante servicios REST o SOAP. De esta forma, no es necesario eliminar los sistemas antiguos, sino que se pueden aprovechar como base, mientras Spring Boot funciona como una capa moderna que mejora la comunicación, el rendimiento y la escalabilidad del sistema.

Finalmente, se puede concluir que Oracle ADF no está obsoleto, pero tampoco es suficiente por sí solo para responder a las necesidades actuales del desarrollo de software. Su mayor valor se obtiene cuando se combina con tecnologías modernas como Spring Boot. Esta integración permite lograr un equilibrio entre la estabilidad y robustez de ADF y la flexibilidad e innovación de Spring Boot, ofreciendo una solución más completa y preparada para el futuro tecnológico de las empresas.

Bibliografía

- **Oracle Corporation. (2023). *Oracle Application Development Framework (ADF) documentation*. https://docs.oracle.com/en/middleware/developer-tools/adf/**

- **VMware. (2024). *Spring Boot reference documentation*. https://docs.spring.io/spring-boot/docs/current/reference/html/**

- **Craig Walls. (2022). *Spring Boot in action* (2nd ed.). Manning Publications.**

- **Mark Heckler. (2021). *Spring Boot: Up and running*. O'Reilly Media.**

- **IBM. (2022). *RESTful web services: Concepts and best practices*. https://www.ibm.com/docs/**